

SPI Master Design & Verification using SystemVerilog

Industry-Style RTL Design Project

Key Features:

- Configurable Data Width
- Configurable SPI Clock Divider
- Support for All SPI Modes (Mode 0, 1, 2, 3)
- CPOL/CPHA Runtime Configuration
- Full-Duplex Data Transfer
- Finite State Machine (FSM) Based Control
- Self-Checking Verification Environment
- SystemVerilog Assertions (SVA)

Project Objective

Design and verify a configurable SPI Master capable of communicating with SPI-compliant peripheral devices while supporting multiple clock modes and scalable transfer widths.

Technologies Used

- SystemVerilog RTL
- Finite State Machine (FSM)
- Shift Registers
- Self-Checking Testbench
- Functional Verification

Designed & Verified By

Sarathi Selvam D

VLSI Design & Verification Portfolio Project

Project Overview

What is SPI?

SPI (Serial Peripheral Interface) is a synchronous serial communication protocol widely used for high-speed communication between a master device and peripheral devices such as sensors, memories, displays, ADCs, and DACs.

SPI enables full-duplex communication, allowing data transmission and reception to occur simultaneously through dedicated data lines.

SPI Communication Signals

- **MOSI** (Master Out Slave In)
Transfers data from the Master to the Slave.
- **MISO** (Master In Slave Out)
Transfers data from the Slave to the Master.
- **SCLK** (Serial Clock)
Generated by the Master to synchronize data transfer.
- **CS_N** (Chip Select - Active Low)
Selects the target Slave device for communication.

Why SPI?

- High-speed communication
- Simple hardware implementation
- Full-duplex data transfer
- Low protocol overhead
- Widely used in embedded and SoC systems

Project Goal

The objective of this project is to design and verify a configurable SPI Master in SystemVerilog that supports:

- Variable data transfer widths
- Configurable clock division
- All four SPI operating modes (CPOL/CPHA)
- Reliable data transmission and reception
- Industry-style verification methodology

Design Specifications

SPI Master Features

The SPI Master is designed as a configurable and reusable RTL IP capable of supporting different SPI peripherals through parameterized design and runtime configuration.

Configurable Parameters

Parameter	Description
DATA_WIDTH	Number of bits transferred in a single SPI transaction
CLK_DIV	Clock divider used to generate the SPI serial clock
CPOL	Clock polarity selection
CPHA	Clock phase selection

Supported SPI Modes

Mode	CPOL	CPHA
Mode 0	0	0
Mode 1	0	1
Mode 2	1	0
Mode 3	1	1

Functional Features

- Full-Duplex Communication
- Master-Controlled Clock Generation
- Active-Low Chip Select (CS_N)
- MSB-First Data Transmission
- Configurable SPI Clock Frequency
- Runtime CPOL/CPHA Configuration
- FSM-Based Transfer Control
- Independent Transmit and Receive Shift Registers

Design Interfaces

Control Interface

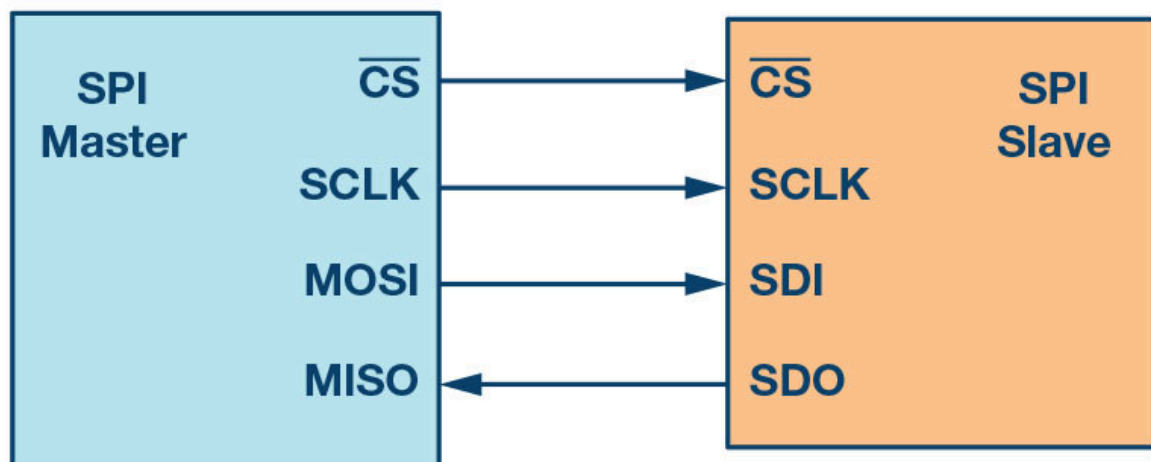
- Start Transfer Signal
- Busy Status Indicator
- Transfer Complete Signal (Done)

Data Interface

- Transmit Data (TX_DATA)
- Receive Data (RX_DATA)

SPI Interface

- MOSI
- MISO
- SCLK
- CS_N



SPI Modes (CPOL & CPHA)

Understanding SPI Timing

SPI communication behavior is controlled by two configuration signals:

CPOL (Clock Polarity)

Determines the idle state of the serial clock (SCLK).

- **CPOL = 0** → SCLK remains LOW when idle.
- **CPOL = 1** → SCLK remains HIGH when idle.

CPHA (Clock Phase)

Determines the clock edge used for data sampling and shifting.

- **CPHA = 0** → Data is sampled on the first clock edge.
- **CPHA = 1** → Data is sampled on the second clock edge.

Supported SPI Modes

SPI Mode	CPOL	CPHA	Idle Clock Level	Data Sampling Edge
Mode 0	0	0	Low	Rising Edge
Mode 1	0	1	Low	Falling Edge
Mode 2	1	0	High	Falling Edge
Mode 3	1	1	High	Rising Edge

Why CPOL and CPHA Matter

Different SPI peripherals operate in different SPI modes. Supporting all four combinations of CPOL and CPHA ensures compatibility with a wide range of devices such as:

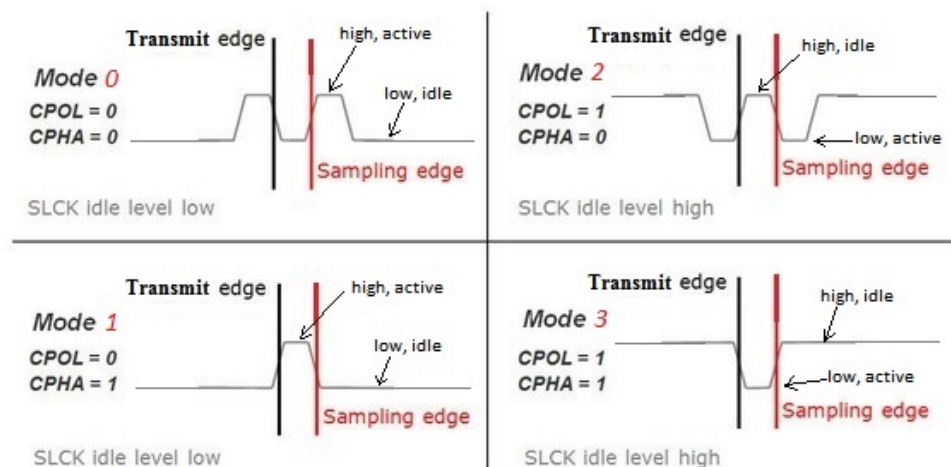
- SPI Flash Memories
- EEPROMs
- ADCs and DACs
- Sensors
- Display Controllers

Implementation in RTL

The SPI Master dynamically selects the sampling and shifting edges based on the CPHA configuration.

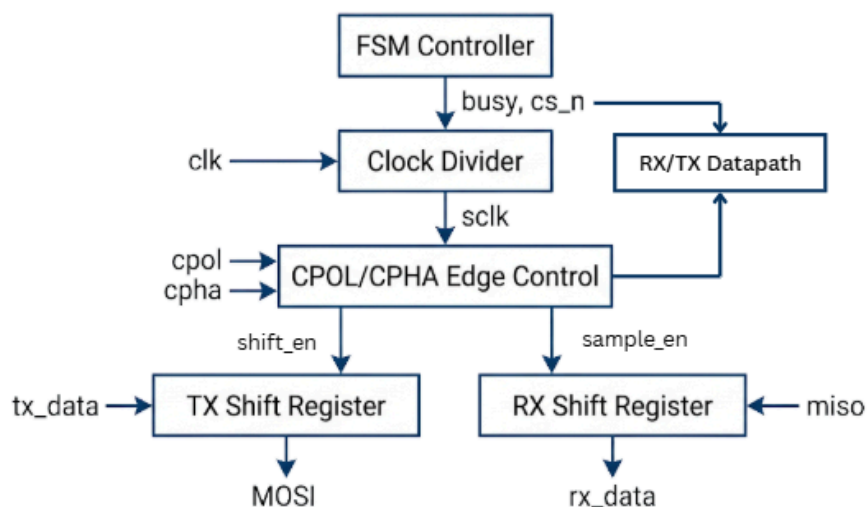
- **Sample Enable (sample_en)** controls when MISO data is captured.
- **Shift Enable (shift_en)** controls when MOSI data is updated.

This allows a single RTL implementation to support all four SPI operating modes.



ARCHITECTURE

SPI MASTER



Clock Generation

SPI is a synchronous communication protocol in which the Master device is responsible for generating the Serial Clock (**SCLK**) used to synchronize data transfer with the Slave device. Since different SPI peripherals support different communication speeds, the SPI Master must provide a configurable clock generation mechanism.

In this design, a **Clock Divider** is used to derive the SPI clock from the system clock. The divider allows the SPI Master to operate at various frequencies while maintaining compatibility with a wide range of SPI devices.

Design Approach

Rather than creating a separate clock domain, the design generates a **clock enable pulse (spi_tick)** using a counter. The SPI clock (sclk) toggles only when this enable pulse is asserted.

Working Principle

1. A divider counter (div_cnt) increments on every rising edge of the system clock.
2. When the counter reaches the programmed divider value, a pulse (spi_tick) is generated.
3. The SPI clock (sclk) toggles whenever spi_tick is asserted.
4. When no transaction is active (busy = 0), sclk returns to its idle state defined by the **CPOL** configuration.

Example

System Clock	Divider Value	SPI Clock
100 MHz	2	25 MHz
100 MHz	5	10 MHz
100 MHz	10	5 MHz

Verification Environment

Objective

The primary goal of verification is to ensure that the SPI Master operates correctly under different configurations and transaction scenarios. A self-checking testbench was developed to automatically validate data transfers and protocol behavior without requiring manual waveform inspection.

Verification Components

Clock Generator

Generates the system clock required by the SPI Master.

Reset Generator

Applies an active-low reset to initialize all internal registers and FSM states.

DUT (Device Under Test)

The SPI Master RTL design being verified.

SPI Slave Model

A behavioral SPI slave model that responds to the master's clock and data transactions, enabling realistic communication testing.

Driver

Applies stimulus to the DUT by initiating SPI transactions and providing transmit data.

Monitor

Observes SPI interface signals (MOSI, MISO, SCLK, and CS_N) and captures transaction activity.

Scoreboard

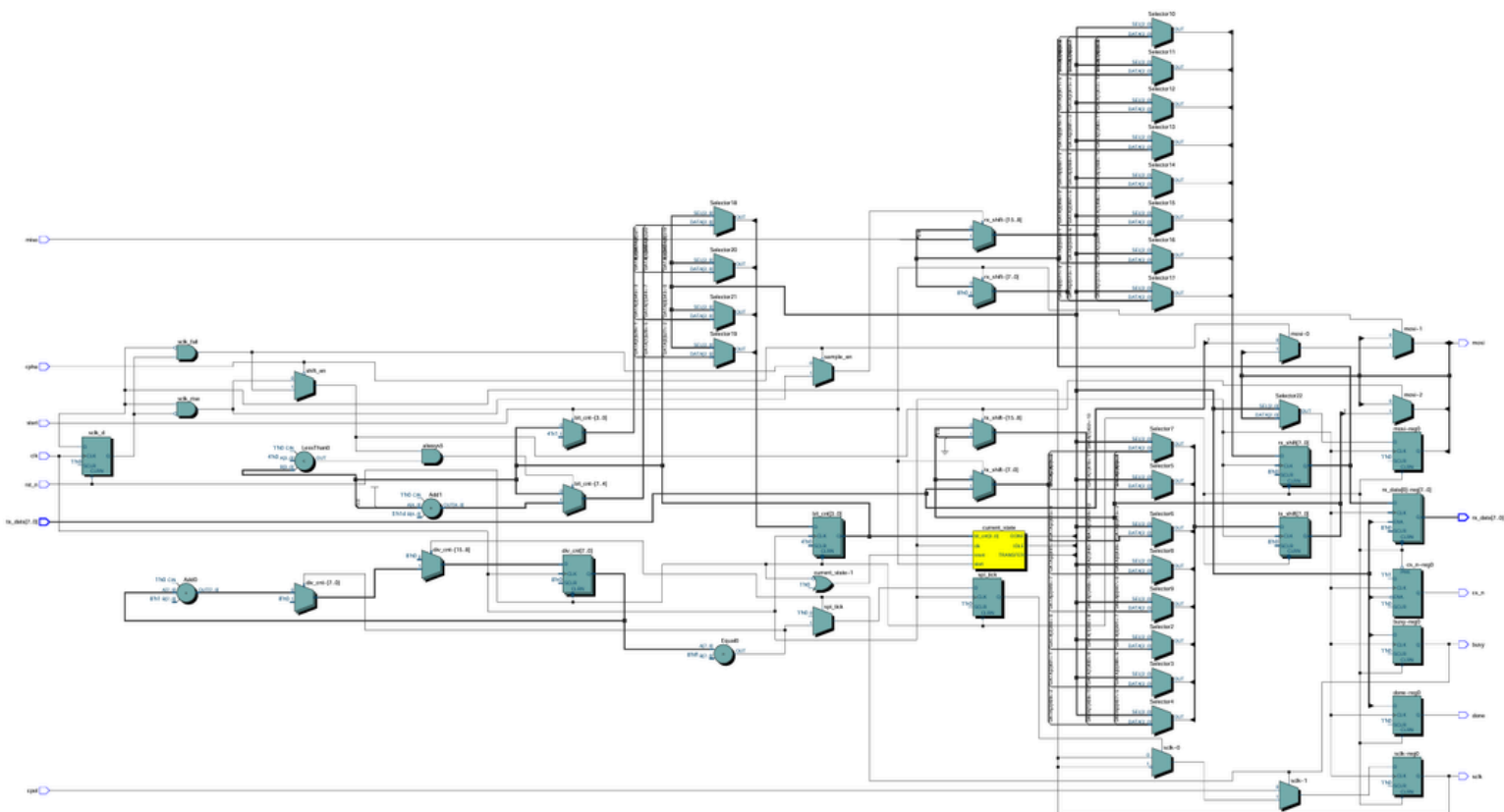
Compares expected data with actual received data and automatically reports pass/fail results.

Self-Checking Methodology

1. Generate a transaction with known transmit and receive data.
2. Drive the transaction to the SPI Master.
3. SPI Slave Model responds with predefined data.
4. Monitor captures the transaction.
5. Scoreboard compares expected and actual results.
6. Test automatically reports PASS or FAIL.

This approach eliminates manual checking and improves verification efficiency.

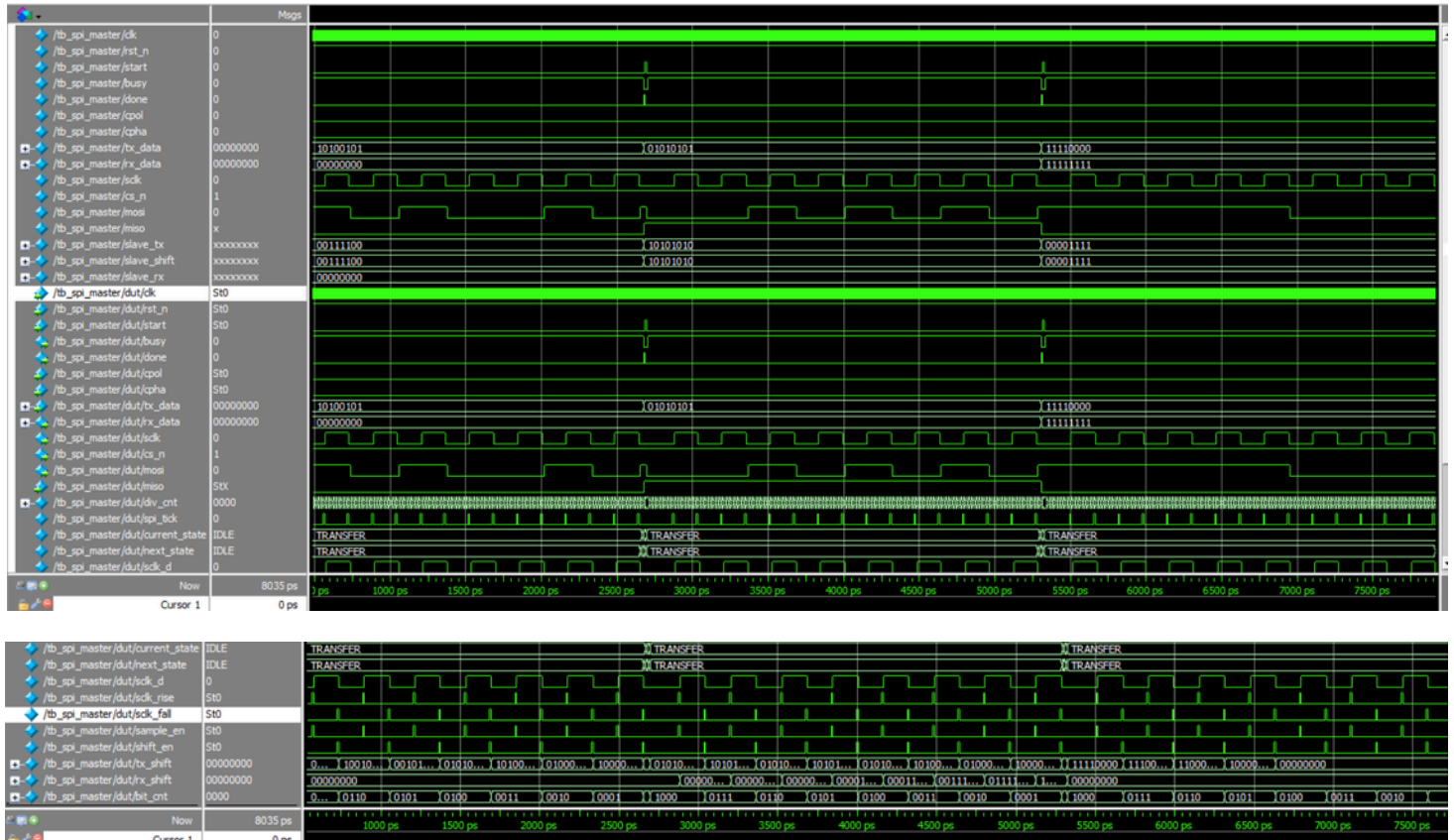
RTL SCHEMATIC



Synthesis Tool: Quartus Prime Lite Edition

Simulation Results

To validate the SPI Master functionality, multiple transactions were executed using a self-checking testbench. The SPI Slave Model provided predefined response data while the SPI Master transmitted and received data through the SPI interface.



Simulation Observation

- The SPI Master successfully initiates transactions when the start signal is asserted.
- CS_N is driven low during active communication and returns high after the transaction completes.
- The generated SCLK operates correctly throughout the transfer period.
- MOSI transmits the loaded tx_data serially in an MSB-first manner.
- The SPI Slave model provides data on MISO, which is correctly captured into the receive shift register.
- sample_en and shift_en are generated according to the configured SPI mode (CPOL=0, CPHA=0).

- The transmit (tx_shift) and receive (rx_shift) registers shift as expected on the selected clock edges.
- The bit counter decrements correctly and indicates completion after the programmed data width is transferred.
- The FSM transitions through **IDLE → TRANSFER → DONE → IDLE**, confirming proper control flow.
- The done signal is asserted at the end of each transaction, indicating successful completion.
- The received data (rx_data) matches the data driven by the slave model, demonstrating correct full-duplex SPI operation.

Result

SPI Master functionality verified successfully for Mode-0 operation (CPOL=0, CPHA=0) with correct clock generation, data transmission, data reception, and FSM behavior.

Key Learnings

SPI Protocol Fundamentals

- Understood the operation of SPI communication using MOSI, MISO, SCLK, and CS_N signals.
- Learned the concept of full-duplex serial communication.

SPI Timing & Modes

- Implemented support for all SPI operating modes using CPOL and CPHA configurations.
- Gained practical understanding of data sampling and shifting edges.

RTL Design Methodology

- Developed a parameterized and synthesizable SPI Master in SystemVerilog.
- Applied modular design principles by separating control and datapath logic.

Finite State Machine (FSM) Design

- Designed and implemented an FSM to manage SPI transaction flow.
- Learned state transition design, control signal generation, and transaction sequencing.

Clock Generation Techniques

- Implemented a configurable clock divider for SPI clock generation.
- Learned industry-standard clock-enable based design instead of creating multiple clock domains.

Shift Register Based Communication

- Implemented transmit and receive shift registers for serial-to-parallel and parallel-to-serial conversion.
- Understood bit-level data movement and synchronization.

Verification Skills

- Developed a self-checking testbench for automated verification.
- Created an SPI Slave model to validate full-duplex communication.
- Analyzed simulation waveforms to debug and verify protocol behavior.

Industry Practices

- Learned RTL coding guidelines using SystemVerilog constructs such as:
 - always_ff
 - always_comb
 - Enumerated FSM states
 - Parameterized design
- Improved documentation and project presentation skills through architecture diagrams and verification reports.

Outcome

Successfully designed, verified, and documented a configurable SPI Master, establishing a strong foundation for future projects involving I2C, APB, AXI, UVM, and RISC-V based SoC design.